Docker & Kubernetes Assignment

1. Docker & Containerization

Q1.1 What is a Docker container and how it is different from a VM?

A docker container is a lightweight environment that runs an application with its dependencies. Unlike a VM, it shares the host OS kernel and starts faster.

Q1.2 Match commands

docker build → build image

docker ps → list running containers

docker run → run container

Q1.3 Three reasons Docker is useful

Environment consistency, fast deployment, easy scalling

Q1.4 True / False

Docker images are mutable False

Containers communicate on networks True

CMD overrides ENTRYPOINT False

Q1.5 Bind mount vs named volume

Bind mount → host folder mapped to container

Named volume → storage managed by Docker

2. Dockerfile and Image Building

Q2.1 Basic Dockerfile for a Python app

Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install –no-cache-dir -r requirements.txt
COPY app.py .
CMD [“python”, “app.py”]
```

➔ docker build -t python-app . → docker run --name python-container python-app

```
PS C:\Users\Robert\Desktop\docker-test> docker build -t python-app .
[+] Building 9.6s (8/8) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 118B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                 1.8s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [1/3] FROM docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a  7.5s
 => => resolve docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a  0.0s
 => => sha256:206356c42440674ecbdf1070cf70ce8ef7885ac2e5c56f1ecf800b758f6b0419 29.78MB / 29.78MB                   3.0s
 => => sha256:2067094444255bb5bf0f8b525f10409691c57f48c48e6bebac8c3a85df4f9dd2 1.29MB / 1.29MB                     1.0s
 => => sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183 14.37MB / 14.37MB                   2.0s
 => => sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a36463b47 10.37kB / 10.37kB                   0.0s
 => => sha256:06fa338add3c2ab8daeff7304a6bb204c68f67743b13553a8999605fc8097c58 1.75kB / 1.75kB                     0.0s
 => => sha256:7c68b568387255c3db2b374da329795e7c25520bd232458282d62d606296e719 5.47kB / 5.47kB                     0.0s
 => => sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2 250B / 250B                         1.5s
 => => extracting sha256:206356c42440674ecbdf1070cf70ce8ef7885ac2e5c56f1ecf800b758f6b0419                           2.2s
 => => extracting sha256:2067094444255bb5bf0f8b525f10409691c57f48c48e6bebac8c3a85df4f9dd2                           0.2s
 => => extracting sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183                           1.7s
 => => extracting sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2                           0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 69B                                                                                    0.0s
 => [2/3] WORKDIR /app                                                                                              0.1s
 => [3/3] COPY app.py .                                                                                             0.0s
 => exporting to image                                                                                              0.1s
 => => exporting layers                                                                                             0.0s
 => => writing image sha256:30a72650c705685d33150f8451499fa9945355897f6e53f69d62917626f96ce0                        0.0s
 => => naming to docker.io/library/python-app                                                                       0.0s
```

```
PS C:\Users\Robert\Desktop\docker-test> docker run python-app
Hello from Docker container
PS C:\Users\Robert\Desktop\docker-test> docker ps
CONTAINER ID   IMAGE                              COMMAND                  CREATED      STATUS         PORTS
                                                                                   NAMES
568efb388e3b   licenta-pubsub-worker              "./worker"               6 days ago   Up 4 minutes   0.0.0.0:8081->80
81/tcp, [::]:8081->8081/tcp                                                        licenta-worker
ae773cec7af1   licenta-pubsub-producer            "./producer"             6 days ago   Up 4 minutes   0.0.0.0:8080->80
80/tcp, [::]:8080->8080/tcp                                                        licenta-producer
5226dc159040   grafana/loki:latest                "/usr/bin/loki -conf…"   6 days ago   Up 4 minutes   0.0.0.0:3100->31
00/tcp, [::]:3100->3100/tcp                                                        licenta-loki
4d52045ec007   grafana/grafana:latest             "/run.sh"                6 days ago   Up 4 minutes   0.0.0.0:3001->30
00/tcp, [::]:3001->3000/tcp                                                        licenta-grafana
398b515fe7d6   rabbitmq:3-management              "docker-entrypoint.s…"   6 days ago   Up 4 minutes   0.0.0.0:5673->56
72/tcp, [::]:5673->5672/tcp, 0.0.0.0:15673->15672/tcp, [::]:15673->15672/tcp   licenta-rabbitmq
07afb49363c5   timescale/timescaledb:latest-pg14  "docker-entrypoint.s…"   6 days ago   Up 4 minutes   0.0.0.0:5433->54
32/tcp, [::]:5433->5432/tcp                                                        licenta-db
PS C:\Users\Robert\Desktop\docker-test>
```

Q2.2 Multi-stage Dockerfile with no pip/build tools in final image

```
# Stage 1 - builder
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Stage 2 - final runtime image
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app.py .
CMD ["python", "app.py"]
```

Primul stage instaleaza pachetele, al doilea stage copiaza doar rezultatul final → imagine eficienta

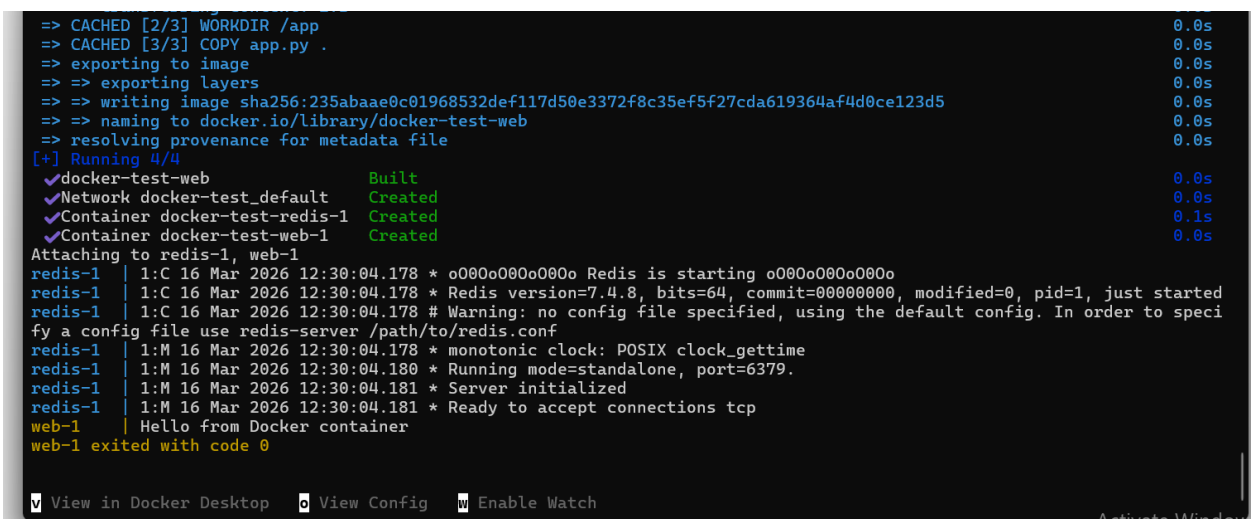
Q2.3 Explain Docker layer caching

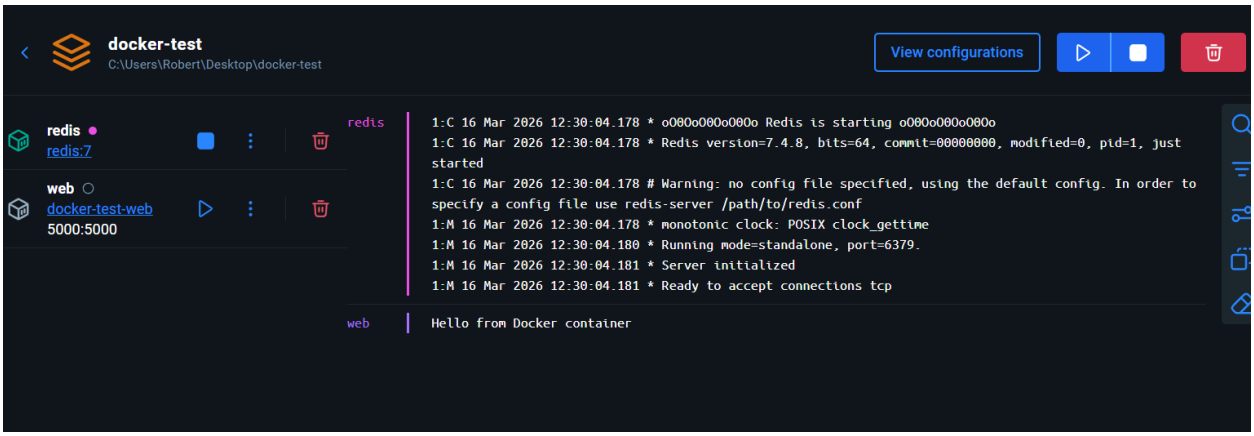
Docker builds images in layers. Each instruction in a Dockerfile, such as FROM, COPY or RUN, creates a new layer. When Docker rebuilds an image, it checks whether a layer has changed. If not, it reuses the cached version instead of rebuilding it.

3. Docker Networking, Volumes and Compose

Q3.1 docker-compose.yml for a Python web app and Redis container

```
version: “3.9”
services:
    web:
        build: .
        container_name: python-web
        ports:
            -“5000:5000”
        depends_on:
            -redis
        environment:
            REDIS_HOST: redis
    redis:
        image: redis:7
        container_name: redis-db
        ports:
            -“6379:6379”
```





- ➔ docker compose up –build (rulez)
- ➔ docker compose down (opresc)

Q3.2 Why use Docker compose?
To run multiple containers together with one command.

Q3.3 Benefits of volume
Data persists, can be backed up, can be shared between containers.

4. Node + PostgreSQL Compose

.env

```
DB_user=postgres
DB_pass=pass123
DB_name=mydb
```

docker-compose.yml

```
version: "3.9"
services:
 app:
  build: .
  ports:
   - "3000:3000"
  env_file:
   - .env
  depends_on:
   - db
 db:
  image: postgres:15
  env_file:
   - .env
```

```
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:
```

5. Kubernetes Core Concepts

1.1 Pod
A Pod is the smallest deployable unit in Kubernetes and runs one or more containers.

1.2 kubelet
kubelet ensures containers run correctly on each node.

1.3 Control plane components
kube-apiserver → API
etcd → cluster data
scheduler → assigns pods to nodes

6. Kubernetes Deployment

nginx-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

```
PS C:\Users\Robert\Desktop\docker-test> kubectl get nodes
NAME             STATUS   ROLES           AGE    VERSIONh
docker-desktop   Ready    control-plane   161d   v1.34.1
PS C:\Users\Robert\Desktop\docker-test> notepad nginx-deployment.yaml
PS C:\Users\Robert\Desktop\docker-test> kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
PS C:\Users\Robert\Desktop\docker-test> kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   0/3     3            0           13s
PS C:\Users\Robert\Desktop\docker-test> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-77bf8679f9-rsmqz   1/1     Running   0          38s
nginx-deployment-77bf8679f9-sbskh   1/1     Running   0          38s
nginx-deployment-77bf8679f9-wrdck   1/1     Running   0          38s
PS C:\Users\Robert\Desktop\docker-test>
```

7. Rolling update

```
nginx-deployment-77bf8679f9-wrdck   1/1     Running   0          38s
PS C:\Users\Robert\Desktop\docker-test> kubectl set image deployment/nginx-deployment nginx=nginx:1.26
deployment.apps/nginx-deployment image updated
PS C:\Users\Robert\Desktop\docker-test> kubectl rollout status deployment/nginx-deployment
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "nginx-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "nginx-deployment" successfully rolled out
PS C:\Users\Robert\Desktop\docker-test>
```

8. Kubernetes Service

nginx-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
  type: ClusterIP
```

➔ kubectl apply -f nginx-service.yaml

```
service/nginx-service created
PS C:\Users\Robert\Desktop\docker-test> kubectl get svc
NAME             TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
kubernetes       ClusterIP   10.96.0.1      <none>        443/TCP   161d
nginx-service    ClusterIP   10.97.100.31   <none>        80/TCP    6s
PS C:\Users\Robert\Desktop\docker-test>
```

9. kube-proxy

kube-proxy routes service traffic to pods.

10. Kubernetes DNS

DNS allows services to be accessed by name.

11. BONUS NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:

  type: NodePort
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

```
PS C:\Users\Robert\Desktop\docker-test> notepad nginx-nodeport.yaml
PS C:\Users\Robert\Desktop\docker-test> kubectl apply -f nginx-nodeport.yaml
service/nginx-nodeport created
PS C:\Users\Robert\Desktop\docker-test> kubectl get svc
NAME             TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes       ClusterIP   10.96.0.1        <none>        443/TCP        161d
nginx-nodeport   NodePort    10.101.221.173   <none>        80:30080/TCP   6s
nginx-service    ClusterIP   10.97.100.31     <none>        80/TCP         57s
PS C:\Users\Robert\Desktop\docker-test>
```

12. Helm

Install repo:

helm repo add bitnami https://charts.bitnami.com/bitnami

helm repo update

Install app:

helm install myredis bitnami/redis

```
NAME    NAMESPACE   REVISION   UPDATED                                   STATUS     CHART          APP VERSION
myredis default     1          2026-03-16 15:01:26.1549457 +0200 EET     deployed   redis-25.3.2   8.6.1
PS C:\Users\Robert\Desktop\docker-test>
```

```
PS C:\Users\Robert\Desktop\docker-test> kubectl get pods
NAME                                  READY   STATUS    RESTARTS        AGE
myredis-master-0                      1/1     Running   0               96s
myredis-replicas-0                    1/1     Running   0               96s
myredis-replicas-1                    1/1     Running   0               33s
myredis-replicas-2                    0/1     Running   0               12s
nginx-deployment-57489d7c8d-fzt7j     1/1     Running   1 (2m20s ago)   28m
nginx-deployment-57489d7c8d-hccvj     1/1     Running   1 (2m20s ago)   28m
nginx-deployment-57489d7c8d-qrmjh     1/1     Running   1 (2m20s ago)   28m
PS C:\Users\Robert\Desktop\docker-test>
```

13. Horizontal Pod Autoscaler

Enable metrics

minikube addons enable metrics-server

Create autoscaler

kubectl autoscale deployment nginx-deployment --cpu-percent=50 --min=2 --max=5

```
PS C:\Users\Robert\Desktop\docker-test> kubectl get hpa
NAME               REFERENCE                    TARGETS              MINPODS   MAXPODS   REPLICAS   AGE
nginx-deployment   Deployment/nginx-deployment   cpu: <unknown>/50%   2         5         0          6s
PS C:\Users\Robert\Desktop\docker-test>
```

14. Simulate Load

kubectl run -it --rm load-generator --image=busybox -- sh

while true; do wget -q -O- http://nginx-service; done

```
PS C:\Users\Robert\Desktop\docker-test> kubectl get pods
NAME                                READY   STATUS    RESTARTS        AGE
load-generator                      1/1     Running   0               33s
myredis-master-0                    1/1     Running   0               3m8s
myredis-replicas-0                  1/1     Running   0               3m8s
myredis-replicas-1                  1/1     Running   0               2m5s
myredis-replicas-2                  1/1     Running   0               104s
nginx-deployment-57489d7c8d-fzt7j   1/1     Running   1 (3m52s ago)   30m
nginx-deployment-57489d7c8d-hccvj   1/1     Running   1 (3m52s ago)   30m
nginx-deployment-57489d7c8d-qrmjh   1/1     Running   1 (3m52s ago)   30m
PS C:\Users\Robert\Desktop\docker-test>
```

15. Monitoring

kubectl describe pod <pod>

kubectl logs <pod>

16. CrashLoopBackOff causes app crash, wrong configuration, missing dependency

17. EKS

Managed node groups → EC2 nodes managed by AWS.

Fargate → serverless pods without managing nodes.

18. IRSA

Steps:

1. create IAM policy
2. enable OIDC provider
3. create service account
4. attach role to pod

19. BONUS 2-tier app

Redis deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7
```

Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: redis-service
spec:
  selector:
    app: redis
  ports:
  - port: 6379
  type: ClusterIP
```

```
PS C:\Users\Robert\Desktop\docker-test> kubectl create deployment redis --image=redis:7
deployment.apps/redis created
PS C:\Users\Robert\Desktop\docker-test> kubectl expose deployment redis --port=6379 --name=redis-service
service/redis-service exposed
PS C:\Users\Robert\Desktop\docker-test> kubectl expose deployment redis --port=6379 --name=redis-service
Error from server (AlreadyExists): services "redis-service" already exists
PS C:\Users\Robert\Desktop\docker-test> kubectl expose deployment frontend --port=80 --type=NodePort --name=frontend-service
Error from server (NotFound): deployments.apps "frontend" not found
PS C:\Users\Robert\Desktop\docker-test> kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           34m
redis              1/1     1            1           27s
PS C:\Users\Robert\Desktop\docker-test> kubectl get svc
NAME               TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes         ClusterIP   10.96.0.1        <none>        443/TCP        161d
myredis-headless   ClusterIP   None             <none>        6379/TCP       5m38s
myredis-master     ClusterIP   10.99.19.70      <none>        6379/TCP       5m38s
myredis-replicas   ClusterIP   10.104.3.2       <none>        6379/TCP       5m38s
nginx-nodeport     NodePort    10.101.221.173   <none>        80:30080/TCP   31m
nginx-service      ClusterIP   10.97.100.31     <none>        80/TCP         31m
redis-service      ClusterIP   10.103.19.89     <none>        6379/TCP       25s
PS C:\Users\Robert\Desktop\docker-test> kubectl create deployment frontend --image=nginx
deployment.apps/frontend created
PS C:\Users\Robert\Desktop\docker-test> kubectl expose deployment frontend --port=80 --type=NodePort --name=frontend-service
service/frontend-service exposed
PS C:\Users\Robert\Desktop\docker-test> kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
frontend           1/1     1            1           21s
nginx-deployment   3/3     3            3           35m
redis              1/1     1            1           78s
PS C:\Users\Robert\Desktop\docker-test>
```